

Even sorting short lists is important to do quickly if it's something that we have to do many times. The world's search engines, for example, collectively receive a trillion searches every few months and have to order each set of results from most to least relevant before delivering them to users. If they can cut the time required for one simple sort from one second to half a second, they cut their required processing time from a trillion seconds to half a trillion seconds. This saves time for users (saving a thousand seconds for half a billion people really adds up!) and reduces data processing costs, and by consuming less energy, efficient algorithms are even environmentally friendly.

The final reason to create faster algorithms is the same reason that people try to do better in any pursuit. Even though there is no obvious need for it, people try to run the 100-meter dash faster, play chess better, and cook a tastier pizza than anyone ever has before. They do these things for the same reason George Mallory said he wanted to climb Mount Everest: "because it's there." It's human nature to push the boundaries of the possible and strive to be better, faster, stronger, and more intelligent than anyone else. Algorithm researchers are trying to do better because, among other reasons, they wish to do something remarkable, whether or not it is practically useful.

Measuring Time Precisely

Since the time required for an algorithm to run is so important, we should be more precise than saying that insertion sort takes a "long time" or "less than a second." How long, exactly, does it take? For a literal answer, we can use the `timeit` module in Python. With `timeit`, we can create a timer that we start just before running our sorting code and end just afterward. When we check the difference between the starting time and the ending time, we find how long it took to run our code.

```
from timeit import default_timer as timer

start = timer()
cabinet = [8,4,6,1,2,5,3,7]
sortedcabinet = insertion_sort(cabinet)
end = timer()
print(end - start)
```
